

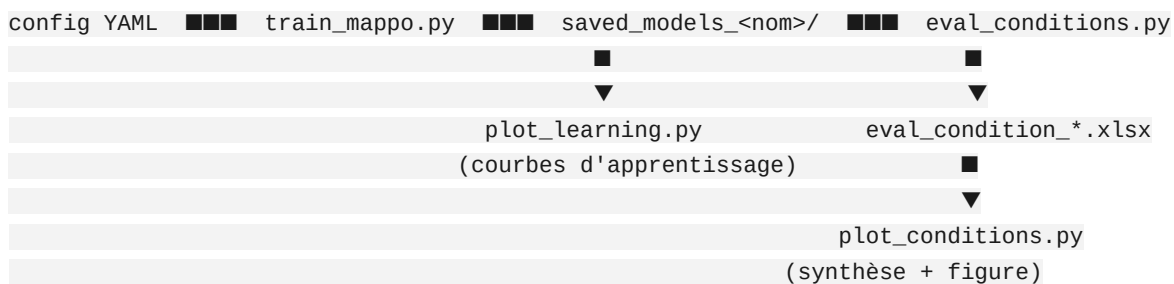
Mode d'emploi des scripts RL Starfarm

Tout se lance depuis `STAR_FARM/rl/`. Un serveur **GAMA headless** doit tourner sur le port visé - c'est la seule dépendance externe, et la cause n°1 des échecs au lancement.

```
gama-headless.bat -socket 1001
gama-headless.bat -socket 1002
```

Deux serveurs permettent deux runs en parallèle. **Un port = un run** : deux processus sur le même port se marchent dessus.

1. La chaîne complète



2. Entraîner

```
py -u train_mappo.py config_mappo3_peragent.yaml
```

Le `-u` n'est pas optionnel si vous redirigez la sortie : sans lui Python met stdout en tampon et le fichier de log reste vide pendant des heures, ce qui rend tout suivi impossible.

Durée : ~2,5 min/épisode sur Dong Thap old simplifiée, ~0,9 min sur Dong Thap simplifiée (même nombre de fermes, géométrie différente). Pour 400 épisodes, comptez 6 à 17 h.

Ce qu'il faut savoir sur la config

Tout se règle dans le YAML, rien dans le code.

```
gama:
  port: 1001          # serveur GAMA vise
  province: Dong Thap # OPTIONNEL - absent = valeur par défaut du GAML
  simple_spatial_data: true # OPTIONNEL - true = carte 10 fermes, false = carte complete
  substep_days: 122   # sous-pas saisonnier
training:
  reward_mode: per_agent # per_agent | pollution | balanced | global
  balance_weight: 20000.0 # lambda, seulement si reward_mode: balanced
  num_episodes: 400
  seed: 0
checkpoint:
  dir: saved_models_mappo3_xxx # UN DOSSIER PAR RUN
  resume: true                 # reprend au dernier checkpoint si le dossier existe
```

province et simple_spatial_data sont optionnels : absents, le modèle prend les valeurs par défaut du GAML. Présents, ils sont relus dans la simulation au démarrage et le run **échoue immédiatement** si GAMA les a ignorés - sans ce garde-fou, 400 épisodes peuvent tourner sur la mauvaise carte sans que rien ne le signale.

dir doit être unique par run. Réutiliser un dossier avec `resume: true` reprend le run précédent au lieu d'en démarrer un neuf ; avec `resume: false` il l'efface.

Attention : la config `pollution` utilise `lr: 0.0001` et `reward_scale: 1.0` là où les autres utilisent `3e-4` et `1e-06`. La pollution vaut $\sim 0,03$, il lui faut sa propre échelle. Si vous dérivez une config pollution, **reprenez ces deux valeurs**.

Ce que produit un entraînement

fichier	contenu
<code>starfarm_ippo.pth / .onnx</code>	dernier checkpoint (le <code>.onnx</code> embarque la normalisation)
<code>best_starfarm_ippo.pth / .onnx</code>	meilleur checkpoint selon l'éval greedy périodique
<code>episode_rewards.json</code>	liste de 400 flottants - le retour sous la récompense du run
<code>eval_returns.json</code>	<code>[{"episode": 24, "return": 316064.9}, ...]</code> tous les 25 épisodes

best_ n'est pas toujours le bon choix. Il retient le maximum d'une série d'évaluations. Si la courbe est stable, il désigne une politique convergée ; si elle oscille (c'est le cas des `lambda` élevés), il désigne l'épisode le plus chanceux. Les rapports utilisent le checkpoint **final** pour les politiques équilibrées, `best_` seulement pour Profit.

3. Évaluer

```
py eval_conditions.py --province "Dong Thap old" --simple --port 1001 \
--tag dtold_simple --episodes 3
```

option	effet
<code>--simple / --full</code>	carte 10 fermes / carte complète (748 ou 992)
<code>--tag NOM</code>	sort dans <code>eval_condition_<NOM>.xlsx</code>
<code>--episodes N</code>	moyenne sur N épisodes (3 sur petite carte, 1 sur grande : ~ 45 min/épisode)
<code>--only "Label"</code>	liste séparée par virgules ; sinon toutes les politiques
<code>--no-baselines</code>	saute les deux stratégies constantes sans IA
<code>--resume</code>	ne rejoue que les politiques absentes du classeur
<code>--stochastic</code>	échantillonne la politique au lieu de prendre sa moyenne

Pour ajouter un modèle à la liste évaluable, ajoutez une ligne à `POLICIES` en tête du fichier : (`"Mon label", "saved_models_mappo3_xxx", "starfarm_ippo.pth"`).

Gardez les baselines. Le profit par ferme n'est pas comparable d'une carte à l'autre : sur `Dong Thap old` la stratégie constante rapporte $+4,7$ k€/ferme, sur `Dong Thap` elle en perd $8,6$. Seul l'écart à la baseline **de la même carte** a un sens.

Le classeur produit contient 5 feuilles : `condition` (quelle carte, combien de fermes), `legende` (à lire en premier), `comparaison` (une ligne par politique), `synthese_fermiers` (une ligne par ferme), `detail` (une ligne par année x ferme).

4. Figures

```
py plot_learning.py      # courbes d'apprentissage de tous les runs -> report/
py plot_conditions.py    # synthese des 4 conditions -> eval_conditions_synthesis.xlsx + png
```

`plot_learning.py` lit la liste `POLICIES` en tête de fichier ; ajoutez-y vos runs, la grille s'adapte au nombre. Le drapeau booléen final indique si le run est sur la carte de référence : mettez `False` pour une autre province, sa courbe passera en pointillés et ne sera pas présentée comme comparable.

`plot_conditions.py` ne lit que les quatre tags canoniques (`dtold_simple`, `dtold_full`, `dtnew_simple`, `dtnew_full`). Les autres classeurs sont ignorés - c'est voulu.

5. Les pièges qui coûtent des heures

La mise en veille tue le socket GAMA. Le client Python reste alors bloqué *indéfiniment* sur une réponse qui n'arrivera jamais : le processus est vivant, il ne consomme pas de CPU, et rien ne le signale. Ça a coûté 17 h une nuit. Avant un run long :

```
powercfg /change standby-timeout-ac 0
```

Ne jugez jamais la santé d'un run à l'existence du processus - jugez-la à sa **sortie**. Un run sain écrit dans son log à chaque épisode et réécrit son classeur à chaque politique. Un fichier dont la date de modification ne bouge plus depuis plus longtemps qu'un épisode est mort, même si le processus est là.

Redémarrez GAMA entre les grosses campagnes. Chaque `load_experiment` laisse une expérience en mémoire ; après quelques dizaines de chargements les serveurs passent de 2,7 à 3,8 Go et ralentissent.

Ne modifiez pas le GAML pendant qu'un run tourne - `env.reset()` relit le fichier à chaque épisode.

Sous PowerShell, Start-Process découpe les arguments contenant des espaces. Il faut quoter deux fois :

```
-ArgumentList 'eval_conditions.py', '--province', '"Dong Thap old"'
```

Deux comptabilités du profit coexistent. Le `greedy_return` affiché à l'entraînement somme le `global_profit` de GAMA ; `eval_conditions.py` somme les récompenses par ferme. Elles ne coïncident pas (Profit : 359 312 contre 461 644 sur la même politique). **Ne comparez jamais un chiffre d'entraînement à un chiffre d'évaluation.**

6. Reprendre un run interrompu

Les entraînements sauvegardent tous les 10 épisodes et `resume: true` repart du dernier checkpoint : relancez simplement la même commande. Pour les évaluations, `--resume` conserve les politiques déjà présentes dans le classeur et ne rejoue que les manquantes - utile quand une condition à 992 fermes s'interrompt après 6 politiques sur 8.

7. Points d'entrée par besoin

besoin	script
entraîner une politique	<code>train_mappo.py <config></code>
comparer des politiques sur une carte	<code>eval_conditions.py</code>
détail ferme par ferme d'une politique	<code>eval_per_farmer.py</code>
variance inter-épisodes d'une politique	<code>eval_variability.py</code>
seulement les stratégies sans IA	<code>eval_baselines.py</code>

frontière profit / pollution du sweep	eval_objectives.py
produire un PDF depuis un .md	report/generate_pdf.py <fichier.md>